

Evaluating Almost-Local Reads (ALRs) in a Raft-based Distributed Key-Value Store

Nafiur Rahman Khadem, Giang Ta, Tuan Le, Sicheng Zhou

December 6, 2025

1 Introduction

While studying distributed systems, we learned about Raft as an understandable consensus protocol that achieves fault tolerance while ensuring linearizable consistency through leader-based replication. However, our classroom exposure was largely theoretical. We wanted hands-on experience building a complete Raft implementation from scratch to understand the practical challenges of implementing consensus protocols, including handling network partitions and message loss, and ensuring correctness under concurrent operations.

Beyond simply implementing the textbook algorithm, we also wanted to explore how production systems adapt Raft for real-world workloads. Modern distributed key-value stores often encounter read-heavy traffic patterns, where the standard Raft design creates a bottleneck: all reads must pass through the leader to ensure linearizability, leaving follower replicas underutilized. This motivated us to investigate read optimization techniques that could make our implementation production-ready for such workloads.

We found inspiration in the LAW theorem [1], which proves that any linearizable asynchronous crash-tolerant protocol cannot provide fully local reads without blocking. The paper introduces Almost Local Reads (ALR) as a practical compromise: followers can serve reads after lightweight synchronization with the leader. In this project, we implement three variants—a baseline leader-only system, full ALR across all replicas, and an intermediate leader-optimized version—to empirically evaluate the trade-offs between more network usage and processing at more replicas. Our goal was to bridge the gap between theory and practice by understanding both how to build Raft correctly and how to optimize it for read-intensive production environments. Our source code is available at <https://github.com/shafinkhadem/raft-kvs>.

2 Background & Motivation

Raft [2] is a consensus protocol designed for understandability and practical deployment. A single elected leader manages log replication to followers, ensuring all replicas apply operations in the same order. This leader-centric design simplifies correctness reasoning but creates a scalability bottleneck: all client operations must go through the leader to guarantee linearizability, even though followers maintain identical state and read-only operations don't really need to get committed through consensus.

The LAW theorem [1] formalizes why this bottleneck exists: no linearizable asynchronous crash-tolerant protocol can provide truly local reads without blocking operations. Almost Local Reads (ALR) offer a practical compromise. Instead of requiring full consensus for each read and processing all reads at the leader, followers can batch process reads after synchronizing with the leader through a lightweight step. The key insight is that followers

submit special sync operations through the Raft log itself — when a sync operation commits and gets executed in a follower’s state machine, the follower knows it has applied all preceding writes and can safely execute batched reads locally. This approach maintains linearizability while distributing read load across replicas.

Our evaluation focuses on empirical questions that theory alone cannot answer: How much throughput improvement does ALR provide under realistic workloads? What latency costs does batching introduce? How does performance scale with cluster size and write ratio? By implementing three variants — baseline leader-only consensus-committed reads, full ALR with follower reads, and an intermediate leader-optimized version, we can quantify these trade-offs and understand the practical implications of optimizing consensus systems for read-heavy production workloads.

3 Design & Implementation

3.1 System Architecture

Our implementation is based on the layered architecture from the MIT labs that separates concerns between application logic, consensus coordination, and distributed replication.

The system consists of four primary layers:

Client Layer: Implements retry logic and leader discovery. Clients maintain a cached view of the current leader and handle redirection when contacting non-leader replicas. The client layer abstracts network failures and leadership changes from application code.

Key-Value Server Layer: Provides application-specific logic including Get and Put operations. Each key maintains a version number that clients must specify during updates. This layer is agnostic to the underlying consensus mechanism.

Replicated State Machine (RSM) Layer: Bridges the KV server and Raft consensus. The RSM receives operations from the KV layer, submits them to Raft for ordering, and waits for consensus. Upon commitment, operations are applied to the state machine in a deterministic order across all replicas.

Raft Consensus Layer: Implements the core distributed consensus protocol including leader election, log replication, commitment, and persistence. The Raft layer treats all operations as opaque commands to maintain generality and reusability across different applications.

This layering enables independent testing and modification of each component. For instance, our ALR optimizations primarily affect the RSM and KV layers without requiring changes to the core Raft implementation.

3.2 Baseline Implementation

Our baseline implementation follows the standard Raft design where all client operations are processed through the current leader. When a client issues a Get request, it is submitted to the leader’s Raft log, replicated to a majority of followers, committed, and only then executed against the state machine. This ensures linearizability since reads always reflect all previously committed writes.

While correct, this approach has performance limitations. Read operations incur the full cost of consensus despite not modifying state. The leader becomes a bottleneck as all read traffic concentrates on a single replica. In read-heavy workloads, this limitation constrains system throughput.

3.3 Almost-Local Read (ALR) Implementation

Our full ALR implementation addresses the leader bottleneck by allowing any replica to serve read requests after establishing a synchronization point through the Raft log. This follows the Lazy-ALR approach described in the LAW theorem paper, which leverages the property that all writes are applied in the same total order across all replicas.

The implementation operates through a mechanism that uses the consensus protocol itself for synchronization:

Read Queueing: When a client sends a Get request to any replica (not just the leader), the request is immediately placed in a local queue rather than being executed. Reads accumulate from multiple concurrent clients across all replicas.

Sync-via-Log Protocol: Each replica runs a background goroutine that monitors its local read queue. When pending reads accumulate, the replica initiates synchronization by submitting a special "sync" Put operation through the normal Raft consensus path. This sync operation contains the replica's identity and the number of queued reads to process. The sync Put is a dummy write that does not modify application state. It serves purely as a synchronization marker in the log.

Batch Execution: When a sync Put commits and is applied to the state machine through Raft's normal operation, the target replica knows that all preceding writes have also been applied locally. At this point, it is safe to execute all queued reads up to the batch size recorded prior to the sync operation. The replica processes these reads directly against its local state and returns results to waiting clients. Other replicas don't process the same sync Put since it was not intended for them.

Version-Based Deduplication: To prevent accidental reprocessing of batches, each replica maintains a sync version number that changes after each batch execution. The sync Put includes this version, and execution only proceeds if versions match. This ensures exactly-once semantics for batch processing even if sync operations are re-delivered due to an unreliable network.

Because sync operations flow through the Raft log like normal writes, a replica executing a read batch is guaranteed to have applied all writes that were committed before the sync operation was initiated. The reads observe a consistent snapshot that includes all prior writes.

3.4 ALR-Leader-Only Variant

We also implemented an intermediate optimization: ALR applied only to the leader. In this variant, the leader can execute reads directly against its state machine after confirming it remains the current leader through a lightweight heartbeat and ensuring it has committed at least an entry in its current term - this is achieved by batching reads with a write.

We expected this variant to provide less throughput improvement than full ALR. It represents a middle ground between baseline and full ALR, and it's also much easier to implement than ALR.

4 Evaluation & Comparison

Figure 1 demonstrates the throughput performance of three Raft implementations across varying write ratios. The ALR-leader-only version shows better performance at low write ratios. This significant improvement occurs because the leader batches read requests instead of committing each read through the raft consensus. The full ALR version performs similarly to the base implementation even at low write ratios, which seems counterintuitive. This occurs because while ALR distributes read processing across all replicas through batched

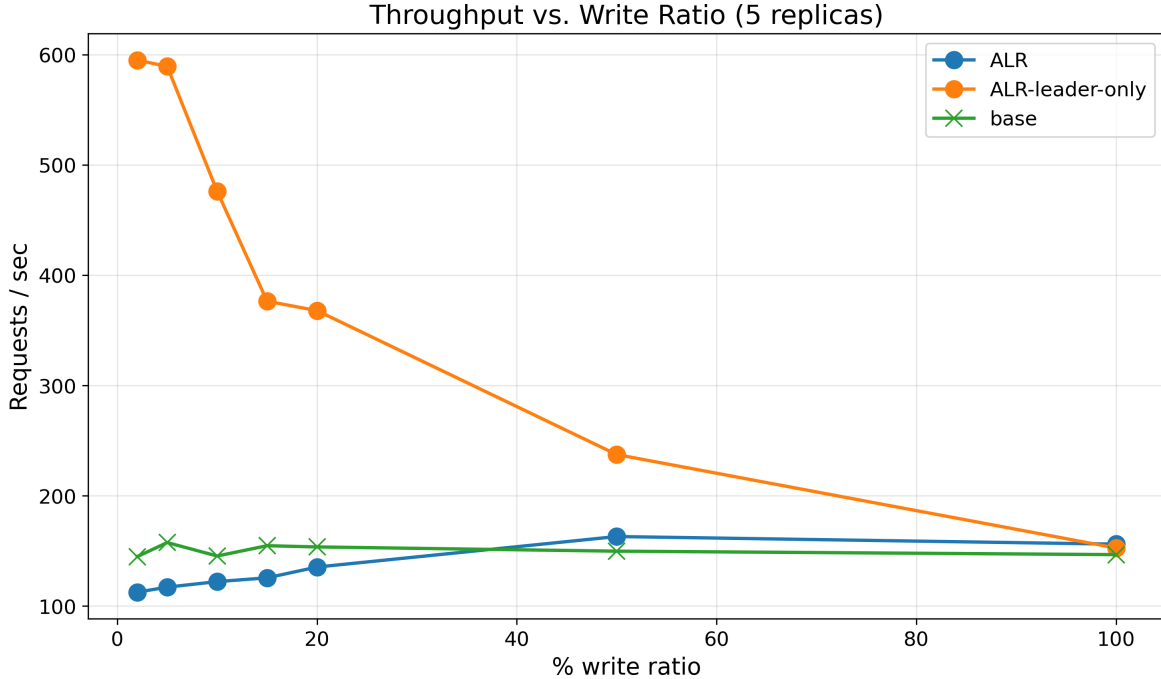


Figure 1: Throughput under different write ratio of different versions of Raft.

sync operations, it incurs additional network overhead: clients send requests to followers, followers submit sync requests to the leader, and responses must propagate back through the follower to the client. This increased network cost offsets the benefits of distributed read processing in our simulated network setting.

Figure 2 compares latency distributions across the three implementations at the 50th, 95th, and 99th percentiles with a 10% write workload. The ALR-leader-only implementation achieves the lowest latencies across all percentiles. These latency characteristics result from the leader processing batched reads locally without requiring additional consensus rounds. Reads piggyback on write operations and naturally satisfy linearizability requirements. The full ALR version exhibits the highest latencies. These elevated latencies, particularly at the tail, result from the additional network hops required in the ALR implementation. Each hop introduces potential queuing delays and network variability. The high P99 latency suggests that under load, some sync operations experience significant delays propagating through this extended path, creating substantial tail latency despite the improved throughput distribution.

Figure 3 shows how each implementation scales as we increase the replication degree from 3 to 7 replicas under two different workloads: 2% writes (left) and 10% writes (right). All of our versions perform worse as replicas increase as the consensus overhead increases. However, we initially expected that under low write ratio, ALR performance would not degrade. But in our simulated network setting, ALR’s higher network overhead dominated any scalability gained by processing reads at follower replicas.

We also evaluated read throughput (without writes) across different concurrent client workloads. As client concurrency increases, longer read batches can be formed. Figure 4 shows that both ALR-leader-only and ALR achieves significantly higher throughput than base version when client parallelism is high. However, ALR still can’t outperform the leader-only version even as we use up to 5000 concurrent clients for 5 server replicas and the read throughput reaches processing bottleneck as we simulate 5 replicas on a single machine.

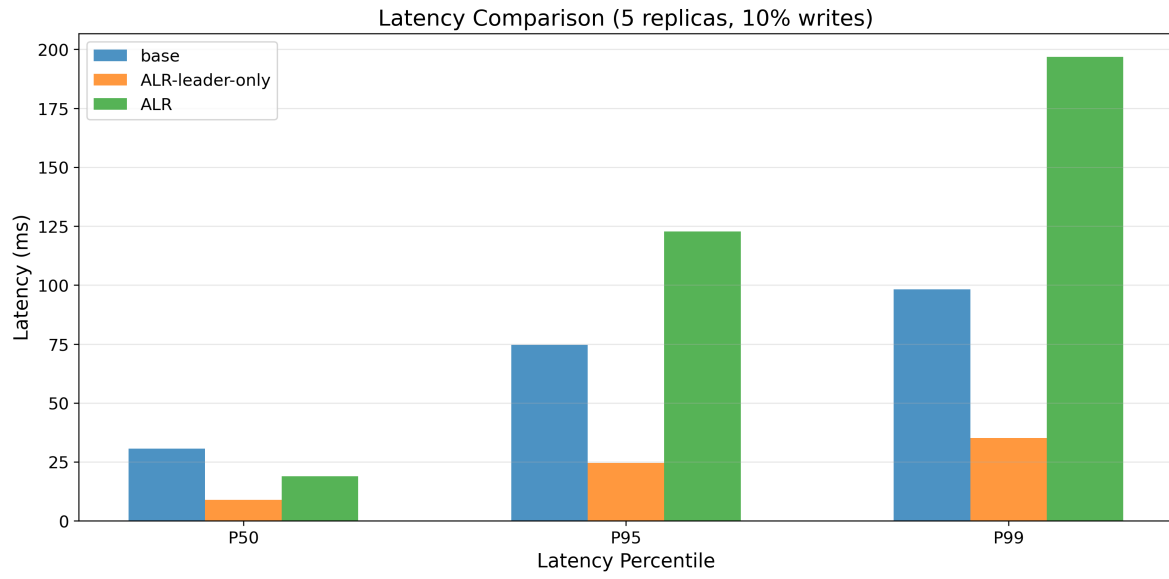


Figure 2: Latency comparisons between different versions of Raft.

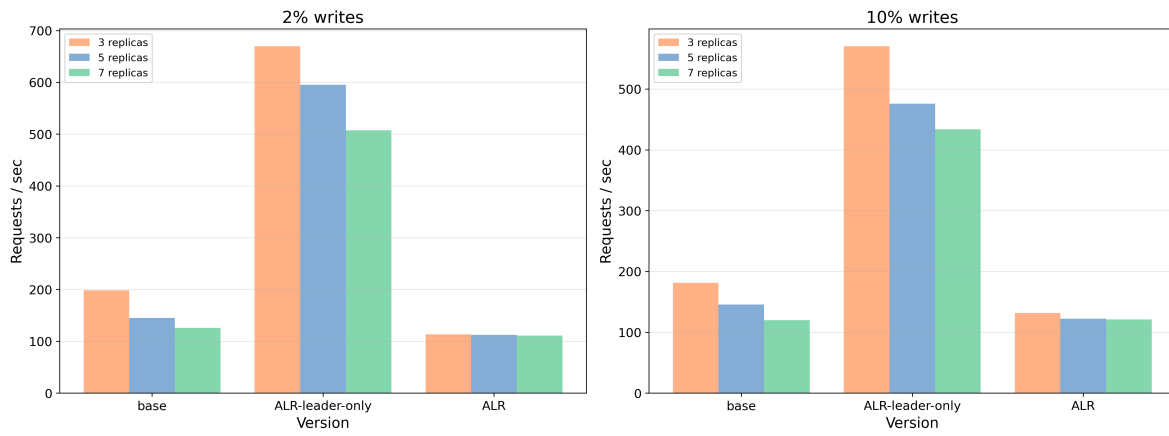


Figure 3: Throughput under different number of replicas.

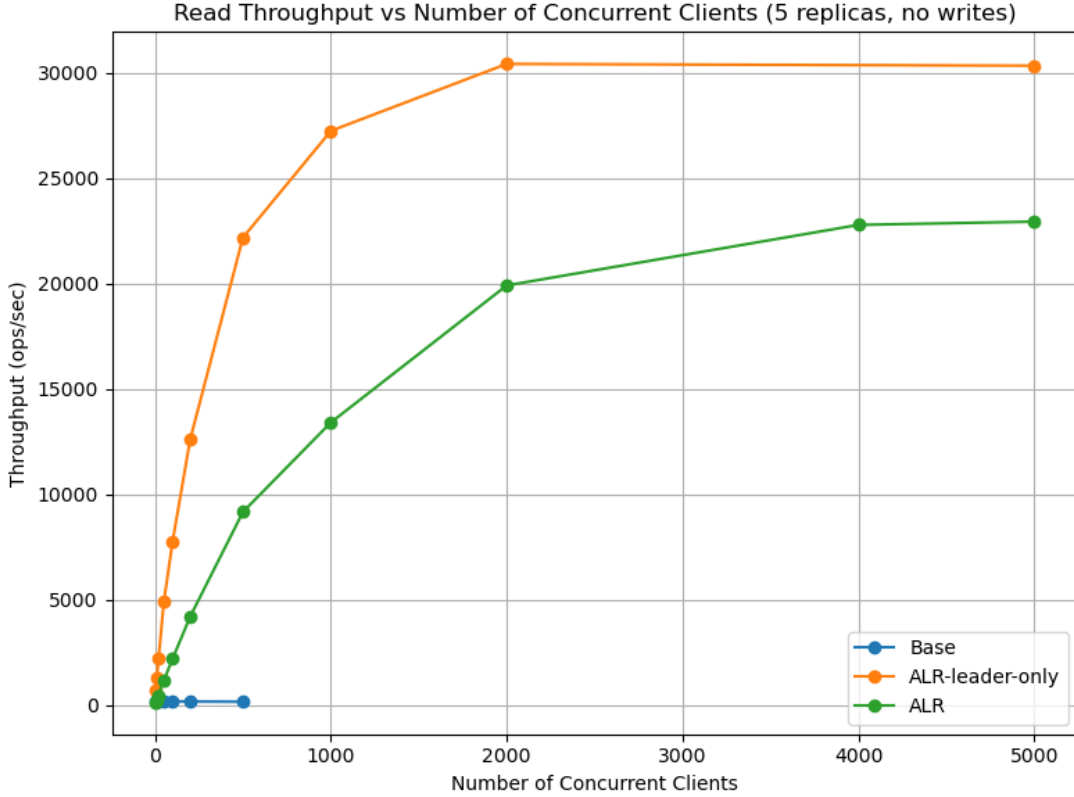


Figure 4: Read throughput with different concurrent client workloads.

5 Lessons Learned

Implementing the Raft algorithm from scratch is difficult, especially because handling diverse failure modes becomes the main challenge. MIT’s simulated RPC network and premade Porcupine test suite helped us save time and improve our confidence in verifying correctness. We found that decoupling at the RPC level proved non-trivial, as setting up production-grade networking for Raft and running our experiment simultaneously proved difficult to balance. Subtle bugs arose from synchronization, duplicate deliveries, response loss etc. Implementing ALR synchronization turned out to be way trickier than leader-only optimization, particularly in terms of ensuring linearizability in spite of an unreliable network.

6 Team Member Contributions

We began with collective research. For the core raft implementation, Sicheng and Nafiur focused on the leader election mechanism, which included establishing the core Raft states, configuration variables, log persistence. Concurrently, Giang and Tuan implemented appending and replicating logs. Nafiur integrated the replicated state machine and the key-value store. Nafiur and Giang parallelly worked on the leader-only optimizations. Later, Nafiur implemented ALR and Sicheng worked on the benchmarks, while Giang tried to set up the system with real RPC, Tuan had prepared an outline for the presentation and report. We also thoroughly reviewed each other’s code, which was essential for debugging and integrating the components correctly.

References

- [1] A. Katsarakis, E. Giortamis, V. Gavrielatos, P. Bhatotia, A. Dragojevic, B. Grot, V. Nagarajan, and P. Fatourou, “The law theorem: Local reads and linearizable asynchronous replication,” *Proceedings of the VLDB Endowment*, vol. 18, no. 9, pp. 2831–2845, 2025.
- [2] D. Ongaro and J. Ousterhout, “In search of an understandable consensus algorithm (extended version),” in *USENIX Annual Technical Conference*, pp. 305–319, 2014.